```
WHERE c.trip_id = t.id;
```

Query the following Trips rows by id and confirm their data column is populated.

```
SELECT * FROM trips WHERE id IN (1,2,3);
```

With the JSON string data populated, you're now able to query it. Select rows where ride_details is present in data by running the following statement in psql. Running the query with EXPLAIN shows that a Sequential Scan of the trips table is being used.

```
EXPLAIN SELECT * FROM trips WHERE data ? 'ride_details';
```

With the data populated, you're now ready to Index it and take advantage of the GIN index type. Create the following GIN index by running the statement in psql and then Analyzing the table.

```
CREATE INDEX ON trips USING gin(data);
```

```
ANALYZE trips;
```

Run the query again with EXPLAIN and notice that a Bitmap Index Scan on trips_data_idx is now used instead of a Sequential Scan.

Now try performing an aggregation. Get the average number of bags in the trunk on trips. This might help inform you whether to send cars with bigger trunks at certain times of the day.

sql/avg_bags_in_trunk.sql

```
SELECT AVG((data->'ride_details'->'bags_in_trunk')::integer)
FROM trips WHERE data ? 'ride_details';
```

Next you'll look at two more indexing options as alternatives to the GIN index you've seen so far.

PgAnalyze describes a containment query (See: *Understanding Postgres GIN Indexes: The Good and the Bad*[7]) example that can be adapted to Rideshare. First add the index, specifying the jsonb_path_ops class in the index creation statement.

```
CREATE INDEX trips_data_path_ops
ON trips USING gin(data jsonb_path_ops);
```

Now, you can answer questions about trips using a containment query like "Which drivers offered a water bottle to Riders?" or "Which trips had two or more bags in the trunk?"

7. https://pganalyze.com/blog/gin-index